

BIOJEPA-AC v0.6: A WORLD MODEL FOR CELLS

Domen Jemec

GPTomics

<https://gptomics.com/>

ABSTRACT

We present BioJEPA-AC v0.6 (Biological Joint-Embedding Predictive Architecture – Action Conditioned), a self-supervised model that predicts how cell states change in response to perturbations. BioJEPA-AC accomplishes this by both learning a unified latent space to represent cell states and learning to move cell state representations in that space based on perturbations (LeCun, 2022). The architecture is built with a transformer-based cell state encoder, a dual-pathway action composer for unified perturbation embeddings, and a cross-attention action-conditioned predictor that shifts cell representation. The AC predictor is conditioned on both genetic and chemical perturbations and can support up to four perturbations per sample. We train on ~ 3.5 million cells from six Perturb-seq (Dixit et al., 2016) datasets covering 10,324 unique perturbations (CRISPRi, CRISPRa, and small-molecule), using a three-stage pipeline: masked latent prediction for the encoder, contrastive alignment for the composer, and action-conditioned prediction with Gaussian NLL for the predictor.

On held-out perturbations, our model achieves a mean sample-level Pearson correlation of 0.898 on the top 20 differentially expressed genes (DEGs), a mean R^2 of 0.860 on the top 50 DEGs, 79.3% directional accuracy on the top 50 DEGs, and a severity correlation of 0.793 (whether the model knows which perturbations have large effects versus small). BioJEPA-AC also uniquely produces per-gene uncertainty estimates, achieving an expected calibration error (ECE) of 0.199. We evaluate across seven categories spanning expression prediction, gene-level analysis, uncertainty calibration, mechanism of action matching, and combination perturbation impact. Because BioJEPA-AC uniquely operates in a shared latent space, it can support diverse downstream tasks through lightweight decoder heads, from expression prediction and directional gene analysis to uncertainty quantification and pathway-level evaluation, while maintaining competitive performance across these differing tasks.

1 MODEL ARCHITECTURE

1.1 OVERVIEW

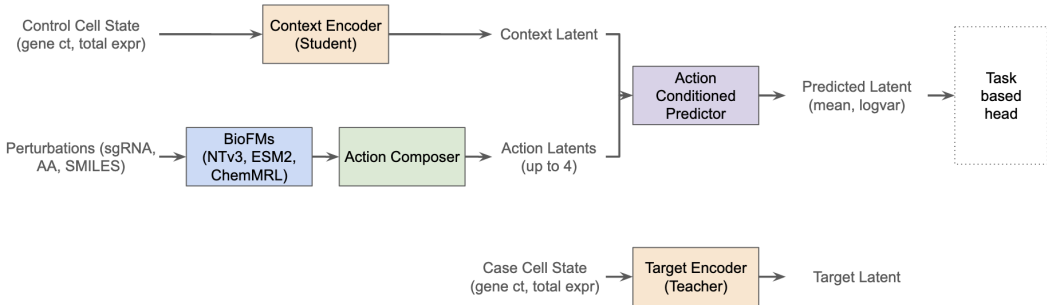


Figure 1: BioJEPA-AC v0.6 inference overview highlighting how a cell state and a perturbation are converted into mean and variance latent representations of the perturbed cell state.

Our v0.6 architecture predicts a latent representation of cell state given its baseline cell state and up to four predefined or novel perturbations across different modalities. We represent cell states across 10,000 genes with continuous-valued expression counts that may have extreme sparsity inherent to Perturb-seq (Dixit et al., 2016) data. The encoder and predictor share core transformer dimensions, identified via Bayesian optimization: 256-dimensional embeddings, 6 layers, 4 attention heads, and a SwiGLU MLP ratio of 4.0. The architecture comprises the following modules:

- **Cell State Encoder:** A transformer-based module that maps cell states into a latent space based on relative gene expression and total cell expression. It serves as both the *Context Encoder* and *Target Encoder*.
- **Action Composer:** A dual-pathway encoder with additive fusion and FiLM-based mode conditioning that creates a unified latent space for the embedding representations of different perturbation types.
- **Action Conditioned Predictor:** A transformer-based module that uses the action latents to adjust the cell state representation in the latent space, generating the latent representation of the perturbed cell state.

1.2 CELL STATE ENCODER

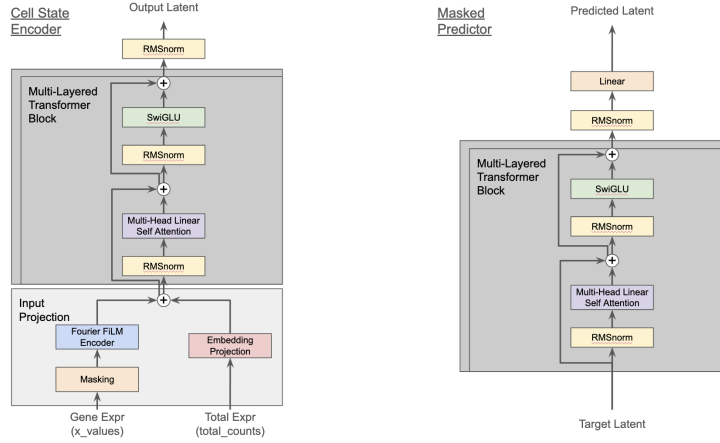


Figure 2: The data that creates our cell state representation in the latent space.

The cell state encoder is the foundation of our joint-embedding architecture, serving as both the context encoder (student) and target encoder (teacher). The encoder is transformer-based with linear attention, SwiGLU (Shazeer, 2020), and RMSNorm. It uses both the count/log normalized expression counts and the sum of total expression to build our cell latent. We encode the total expression count using Gaussian Fourier features (scale=2.38) and apply FiLM conditioning (linear scale=0.81) for expression-dependent modulation. We include the total expression sum to ensure the model can flag unviable cells that would look like “noisy” expression if we only used the normalized per-gene expression. Our cell representation lives in the [token, embedding] space, where tokens correspond to genes. We designed the architecture so we can expand to non-gene-based tokens. Review our explainer¹ notebook for more detail.

1.3 ACTION COMPOSER

The action composer is a dual-pathway linear encoder that projects perturbation embeddings into a shared latent space via separate linear projections, fuses them, and applies FiLM (Perez et al., 2018) conditioning from a learned mode embedding to encode the perturbation with mechanism awareness. Our perturbation representation ends in the [n_perts, 320] space, with a mode embedding dimension of 64. We identified these dimensions via Bayesian optimization.

¹https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_cell_state_encoder.ipynb

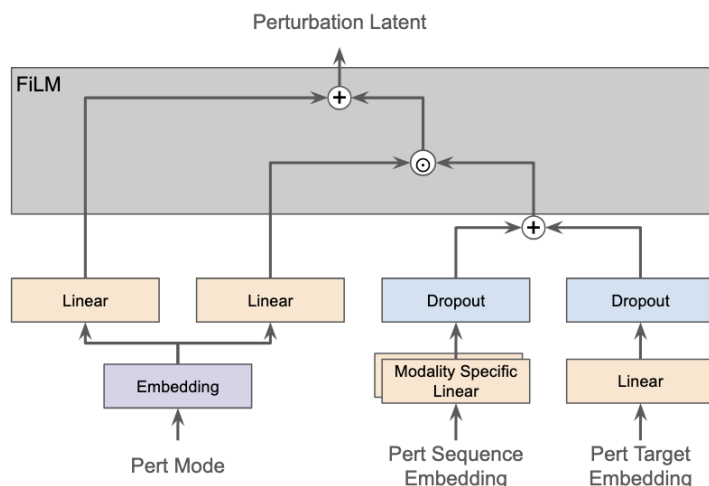


Figure 3: An overview of how we merge different types of perturbations with different data into the same perturbation representation.

Perturbations can be targeted genetic (e.g. CRISPR) perturbations or therapeutics including nucleic acids, proteins, and small molecules. We require that a perturbation has a sequence and/or target (at least one) and a mode (crispr, crispra, overexpression, knockout, inhibitor, agonist, degrader, binder, unknown). To improve flexibility and generalization, we pass the raw sequence and target representations through bio foundation models (Dalla-Torre et al., 2024; Lin et al., 2023) during data preparation rather than feeding them directly to the composer. We rely on the action composer to learn embeddings that represent functionally similar but sequence-different perturbations, enabling expansion to unseen perturbations. Review our explainer² notebook for more detail.

1.4 ACTION CONDITIONED PREDICTOR

The action conditioned predictor is the foundation of the “AC” portion of our BioJEPAC model. The predictor is transformer-based with similar linear attention, SwiGLU, and RMSNorm. It differs from the cell state encoder by employing two different attention layers: cross-attention to shift the cell state based on the perturbation, and self-attention to allow the cell state to learn from itself. The predictor uses target indices to predict only a portion of the cell state representation, similar to how masked models operate. It fuses the unperturbed cell representation from the context encoder with the target tokens, then uses cross-attention to incorporate the action latents from the action composer. The predictor outputs a predicted representation of the perturbed cell state including an uncertainty estimate, each in the [token, embedding] space. It operates in the same shared latent space as the cell state encoder, learning where to move the representation based on the perturbations.

Review our explainer³ notebook for more detail.

2 EVALUATIONS

2.1 FULL MODEL EVALUATION

BioJEPAC’s primary benefit is not creating the joint-embedding space, but being able to take actions, in our case perturbations, and move cell representations in that embedding space. To evaluate whether our latent space and our ability to move representations across it are useful, we use a series of evals both directly on the latent space and with lightweight decoder heads.

²https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_action_composer.ipynb

³https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_ac_predictor.ipynb

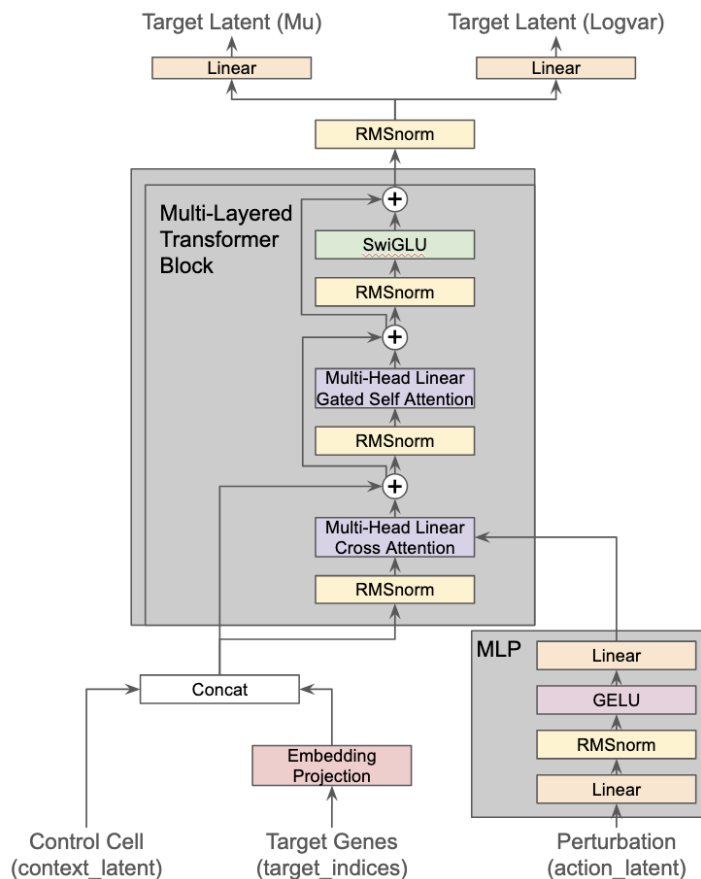


Figure 4: An overview of how we shift the cell state in the shared latent based on the perturbation.

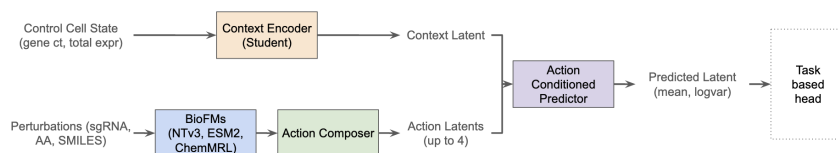


Figure 5: Forward pass of our action conditioned network

To run evals, we take a control cell state, add up to four perturbation representations, and do a forward pass through BioJEPA-AC, generating a predicted perturbed cell state representation that we then pass to the task-specific eval. We run seven evals on our fully trained model.

2.2 EVAL HEADS

Since BioJEPA-AC is an encoder, its output is an embedding, not a directly interpretable value. To evaluate biological utility, we add decoder heads on top but keep them intentionally simple (a single linear layer each) so they don't mask issues in the base model. We use two heads: a linear expression decoder and a linear classifier.

2.2.1 LINEAR EXPRESSION DECODER

Since our cell state latent representation is $[n_genes, embed_dim]$, the decoder projects each gene's embedding to a scalar expression value through its single linear layer. Our model outputs both a

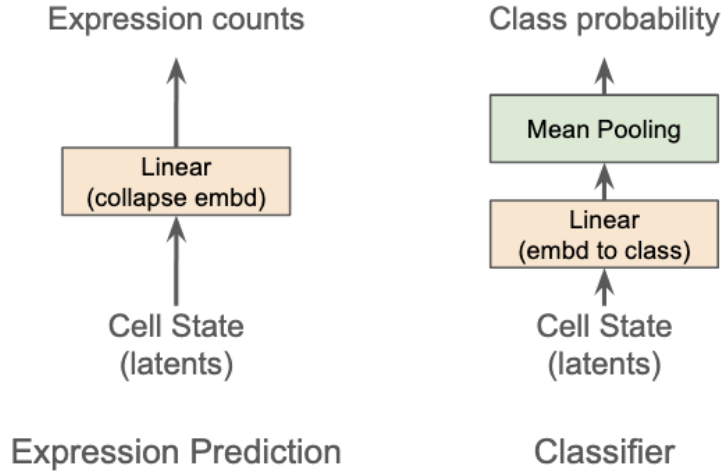


Figure 6: An overview of the different heads, an expression predictor and a general linear classifier, we use to conduct evaluation benchmarking on our BioJEPAC model

mean and log-variance (logvar) representation, but we feed only the mean into this decoder, reserving the logvar for uncertainty analysis.

2.2.2 GENERAL LINEAR CLASSIFIER

We also use a general-purpose linear classifier for predicting input metadata and classes (e.g. cell batch information, cell types, perturbation status, modes, pathways) from latent representations. The classifier projects the latent to the class dimension through a single linear layer, producing raw logits per class. In some cases like classifying a cell latent to a cell type, we add a mean-pooling step to collapse per-gene predictions to per-cell without giving the model more intelligence. We leave outputs as raw logits since our different evals post-process them in different ways.

2.3 EVAL DATA SPLITS – HOLD OUT

To create our hold-out datasets, we target an 85% / 5% / 10% train / val / test split, with actual percentages varying slightly by dataset due to cross-dataset perturbation overlap. We use the GEARS python package⁴ (Roohani et al., 2024) to identify the held-out train / val / test split for Replogle K562 essential (Replogle et al., 2022). This dataset uses a 67.5% / 7.5% / 25.0% train / val / test split, which we adopt as our starting point. The split identifies unique perturbations to hold out. Starting from this, we build a hold-out perturbation list that ensures we hit 15% held-out perturbations per dataset and that those perturbations are held out across all datasets. This ensures we have no perturbation leakage, and allows a direct head-to-head comparison on the Replogle K562 essential dataset using the GEARS-defined held-out set.

For the remaining datasets, where we extracted perturbations at random, we compare against published numbers. Where feasible, we plan to re-run comparison models on our splits.

2.4 EXPRESSION PREDICTION

We use this eval to measure how well our model predicts post-perturbation gene expression. High accuracy here provides interpretable insight into perturbation effects on viability and gene networks. Since most of our 10,000 genes will not change significantly for any given perturbation, predicting no change can be highly accurate. To avoid this trap, we run analyses at multiple granularities including the top 20 and top 50 differentially expressed genes.

⁴<https://github.com/snap-stanford/GEARS>

While we’ve done a number of different analyses, we cover the following commonly performed evals. A major component of our expression benchmark is not looking at the raw prediction made by the linear expression decoder, but the relative prediction to the predicted control. Our baseline *real delta* is the observed change in expression: $\delta_g = x_g^{\text{case}} - x_g^{\text{ctrl}}$. To get our *predicted delta*, we pass both the predicted and control latent representations through the linear expression decoder and compute the difference: $\hat{\delta}_g = \hat{x}_g^{\text{case}} - \hat{x}_g^{\text{ctrl}}$. We use the predicted control expression to isolate our expression prediction into terms of BioJEPAC’s learned latent space, ensuring that if the model learns a different baseline for the control but knows the distance to move the case cell, it can still score well. The expression deltas only work for some of our evals while others rely on absolute prediction. For our *real absolute* expression, we use the observed case expression directly. For our *predicted absolute*, we add our predicted delta to our real control: $\hat{x}_g^{\text{abs}} = x_g^{\text{ctrl}} + \hat{\delta}_g$. These four values, along with our control, form the basis of our evaluation benchmark.

For each of the following evaluations, we explain how the calculation is done and BioJEPAC’s performance by dataset. For a detailed breakdown on how the evaluations are calculated, see our explainer notebook.⁵

2.4.1 SAMPLE LEVEL EVALUATION

We run evaluations at the sample level, calculating per sample in the test set and then aggregating the per-sample results. We build our corpus by pairing case and control cells together based on the batch identifiers in a dataset. That pairing, along with the perturbation information, creates our unique sample.

Sample Level Pearson’s Correlation Coefficient For Top 20 DEGs. For each sample, we calculate Pearson’s r between the *predicted delta* and *real delta* on the top 20 differentially expressed genes (DEGs), selecting the genes with the largest absolute *real delta*. We use $K = 20$ at the sample level since individual samples have noisier delta estimates than perturbation-level means. Even if the predicted magnitudes are off, a high correlation indicates our model has learned the correct expression profile. We calculate:

$$\text{Pearson } r_{\text{sample}} = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{k=1}^K (\hat{\delta}_{i,k} - \bar{\hat{\delta}}_i)(\delta_{i,k} - \bar{\delta}_i)}{\sqrt{\sum_{k=1}^K (\hat{\delta}_{i,k} - \bar{\hat{\delta}}_i)^2} \cdot \sqrt{\sum_{k=1}^K (\delta_{i,k} - \bar{\delta}_i)^2}} \quad (1)$$

Where $K = 20$ genes are selected per sample as the largest $|\delta_{i,k}|$.

Table 1: Sample-level Pearson correlation on top 20 DEGs.

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.8980</i>
Adamson	0.8820
Replogle K562 essential	0.8856
Replogle K562 genome-wide	0.9097
Sciplex	0.8809

2.4.2 PERTURBATION LEVEL EVALUATION

We run evaluations at the perturbation level, defining a unique perturbation as a distinct combination of sequence, target, modality, mode, and cell type. We first compute our expression values (*real delta*, *predicted delta*, *real absolute*, *predicted absolute*, *control absolute*) at the sample level, then take the mean across all samples for each perturbation.

Perturbation Level Mean Pearson Correlation Coefficient on Absolute Expression. We calculate Pearson’s correlation per perturbation on absolute expression values across all genes. Because

⁵https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_eval_expr_prediction.ipynb

we run on all genes, the baseline expression pattern dominates this metric, measuring how well our model learns which genes a perturbation has minimal impact on:

$$\text{Pearson } r_{\text{pert}} = \frac{1}{P} \sum_{p=1}^P \frac{\sum_{k=1}^K (\hat{y}_{p,k} - \bar{\hat{y}}_p)(y_{p,k} - \bar{y}_p)}{\sqrt{\sum_{k=1}^K (\hat{y}_{p,k} - \bar{\hat{y}}_p)^2} \cdot \sqrt{\sum_{k=1}^K (y_{p,k} - \bar{y}_p)^2}} \quad (2)$$

Where $\hat{y}_{p,k}$ and $y_{p,k}$ are the predicted and real absolute expression for perturbation p , gene k , already averaged over all samples of that perturbation. The bars ($\bar{\hat{y}}_p$, $\bar{y}_p$) are means across genes for centering within each perturbation's Pearson computation. Three key differences from the sample-level calculation: we use absolute expression instead of deltas, perturbation means instead of individual samples, and all genes instead of the top-20 DEGs.

Table 2: Perturbation-level mean Pearson correlation on absolute expression.

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.9904</i>
Adamson	0.9843
Replogle K562 essential	0.9848
Replogle K562 genome-wide	0.9917
Sciplex	0.9736

Perturbation Level Mean Pearson Correlation Coefficient on Expression Delta. We also calculate Pearson's correlation per perturbation on expression deltas for all genes:

$$\text{Pearson } \Delta r_{\text{pert}} = \frac{1}{P} \sum_{p=1}^P \frac{\sum_{k=1}^K (\hat{\delta}_{p,k} - \bar{\hat{\delta}}_p)(\delta_{p,k} - \bar{\delta}_p)}{\sqrt{\sum_{k=1}^K (\hat{\delta}_{p,k} - \bar{\hat{\delta}}_p)^2} \cdot \sqrt{\sum_{k=1}^K (\delta_{p,k} - \bar{\delta}_p)^2}} \quad (3)$$

Where $\hat{\delta}_{p,k}$ and $\delta_{p,k}$ are the predicted and real expression deltas for perturbation p , gene k , already averaged over all samples of that perturbation. The bars ($\bar{\hat{\delta}}_p$, $\bar{\delta}_p$) are the means across genes, used for centering within each perturbation's Pearson computation.

The thousands of unchanged genes where prediction and ground truth share the same control baseline can inflate Pearson on absolute expression. Pearson on deltas strips that shared signal, forcing correlation to come entirely from correctly predicting the perturbation effect. However, deltas have lower variance and are centered near zero, so the centering step has less to work with and the few changed genes have outsized influence, making this a harder metric.

Table 3: Perturbation-level mean Pearson correlation on expression delta.

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.3512</i>
Adamson	0.2239
Replogle K562 essential	0.3191
Replogle K562 genome-wide	0.3707
Sciplex	0.2490

Perturbation Level Mean Coefficient of Determination R^2 For Top 50 DEGs. We calculate R^2 between our real and predicted absolute expression values on the top 50 differentially expressed genes for each perturbation, measuring how much variance in the real expression our predictions capture for the most affected genes.

$$R_{\text{top } 50}^2 = \frac{1}{P} \sum_{p=1}^P \left(1 - \frac{\sum_{k \in \mathcal{D}_p} (\hat{y}_{p,k} - y_{p,k})^2}{\sum_{k \in \mathcal{D}_p} (y_{p,k} - \bar{y}_p)^2} \right) \quad (4)$$

Where $\mathcal{D}_p$ is the set of 50 genes with the largest $|\delta_{p,k}|$ for perturbation p , and $\bar{y}_p = \frac{1}{|\mathcal{D}_p|} \sum_{k \in \mathcal{D}_p} y_{p,k}$. Note that while the top-50 genes are selected by delta magnitude, R^2 itself is computed on absolute expression values. Since we already have the linear correlation of all genes calculated using Pearson’s coefficient, we focus R^2 on just the top differentially expressed genes, which tests whether our model accurately captures the magnitude of expression changes at the genes most affected by the perturbation. Unlike Pearson’s correlation, R^2 penalizes magnitude errors, so echoing the control state is not enough.

Table 4: Perturbation-level mean R^2 on top 50 DEGs.

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.8600</i>
Adamson	0.6815
Replogle K562 essential	0.8132
Replogle K562 genome-wide	0.8699
Sciplex	0.8930

2.4.3 CROSS-PERTURBATION EVALUATION

We run evaluations across our perturbation expression profiles. While these evaluations still look at the expression prediction mean per perturbation, they evaluate the predictions against one another to test whether our model can distinguish between different perturbations.

Cross Perturbation Centroid Accuracy. We compare each predicted delta vector against all real delta vectors to find the nearest match. For this calculation, we change how we define a unique perturbation. Since this metric cares mainly about perturbation effect, we re-group our perturbations by taking the mean expression delta across perturbations sharing the same target protein (or sequence if target is unavailable), mode, and cell type. This avoids asking the model to differentiate between different perturbations targeting the same protein. We calculate centroid accuracy as:

$$\text{Centroid Accuracy} = \frac{1}{G} \sum_{g=1}^G \mathbb{I} \left[\arg \min_h \|\hat{\delta}_g - \delta_h\|^2 = g \right] \quad (5)$$

Where G is the number of unique perturbation target groups, and $\hat{\delta}_g, \delta_g$ are gene-length vectors of mean predicted and real expression deltas for group g . This metric becomes harder with more unique perturbations, especially when multiple perturbations share a similar target mechanism.

Table 5: Cross-perturbation centroid accuracy.

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.5967</i>
Adamson	0.1818
Replogle K562 essential	0.0699
Replogle K562 genome-wide	0.6947
Sciplex	0.5128

Cross Perturbation Severity Correlation. We measure whether our model accurately predicts the overall strength of each perturbation’s effect. Using the same perturbation-level mean deltas, we compute the L2 norm (Euclidean magnitude) of both the real and predicted expression delta vectors, reducing each to a scalar severity score. We then calculate Pearson’s correlation across perturbations

between these severity scores:

$$\begin{aligned}
s_p &= \|\delta_p\|_2 = \sqrt{\sum_{k=1}^K \delta_{p,k}^2} \\
\hat{s}_p &= \|\hat{\delta}_p\|_2 = \sqrt{\sum_{k=1}^K \hat{\delta}_{p,k}^2} \\
\text{Pearson } \Delta r_{\text{severity}} &= \frac{\sum_{p=1}^P (\hat{s}_p - \bar{\hat{s}})(s_p - \bar{s})}{\sqrt{\sum_{p=1}^P (\hat{s}_p - \bar{\hat{s}})^2} \sqrt{\sum_{p=1}^P (s_p - \bar{s})^2}}
\end{aligned} \tag{6}$$

Where s_p and $\hat{s}_p$ are the real and predicted severity scores for perturbation p , computed as the L2 norm across all K genes. A high severity correlation indicates our model knows which perturbations have large effects versus small, even if it doesn't perfectly predict the per-gene pattern.

Table 6: Cross-perturbation severity correlation.

Dataset	BioJEPA-AC v0.6
<i>Overall</i>	<i>0.7932</i>
Adamson	0.7412
Replogle K562 essential	0.5273
Replogle K562 genome-wide	0.8435
Sciplex	0.4805

2.5 GENE LEVEL ANALYSIS

We use this eval to assess whether our model correctly identifies which genes are affected by a perturbation and in which direction. Unlike expression prediction, which measures magnitude accuracy, gene-level analysis focuses on categorical behavior across differentially expressed genes. These evaluations rely on the same perturbation-level mean *real delta* and *predicted delta*.

For each of the following evaluations, we explain how the calculation is done and BioJEPA-AC's performance by dataset. For a detailed breakdown on how the evaluations are calculated, see our explainer notebook.⁶

Directional Accuracy For Top 50 DEGs. We measure whether our model correctly predicts if a differentially expressed gene is up-regulated or down-regulated. We first classify each gene's expression delta into one of three categories using a threshold:

$$d(\delta_k) = \begin{cases} +1 & \text{if } \delta_k \geq \tau \\ -1 & \text{if } \delta_k \leq -\tau \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

Where $\tau = 0.25$ is the direction threshold. We apply this classification to both our real and predicted perturbation-level mean expression deltas, then select the top 50 DEGs per perturbation based on the largest absolute real expression delta. We then compare how many of those genes the prediction classified correctly:

$$\text{Direction Accuracy}_{\text{topK}} = \frac{1}{P \cdot K} \sum_{p=1}^P \sum_{k \in \mathcal{T}_p} \mathbb{I}[d(\hat{\delta}_{p,k}) = d(\delta_{p,k})] \tag{8}$$

Where $\mathcal{T}_p$ is the set of $K = 50$ genes with largest $|\delta_{p,k}|$ for perturbation p . Since we select the top 50 out of 10,000 genes, we typically focus on genes with large expression changes, but perturbations with lower overall cell impact and fewer significantly moving genes make this a harder metric.

⁶https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_eval_gene_analysis.ipynb

Table 7: Directional accuracy on top 50 DEGs.

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.7929</i>
Adamson	0.6636
Replogle K562 essential	0.3813
Replogle K562 genome-wide	0.7224
Sciplex	0.7422

2.6 UNCERTAINTY CALIBRATION

We use this eval to assess whether our model’s uncertainty estimates are meaningful. BioJEPAC outputs both a mean and log-variance latent representation for the perturbed cell state. We average the log-variance across genes per sample as our uncertainty signal, and compute MSE between the *predicted delta* and *real delta*, also averaged across genes:

$$\begin{aligned}
 u_s &= \frac{1}{K} \sum_{k=1}^K \text{logvar}_{s,k} \\
 e_s &= \frac{1}{K} \sum_{k=1}^K (\hat{\delta}_{s,k} - \delta_{s,k})^2
 \end{aligned} \tag{9}$$

Where u_s and e_s are the scalar uncertainty and MSE for sample s , averaged across all K genes. A well-calibrated model should produce higher uncertainty for samples where it makes larger errors. These evals currently perform poorly, and we are exploring training and post-training techniques to improve calibration.

For each of the following evaluations, we explain how the calculation is done and BioJEPAC’s performance by dataset. For a detailed breakdown on how the evaluations are calculated, see our explainer notebook.⁷

Expected Calibration Error (ECE). We measure calibration using ECE, which quantifies the gap between predicted uncertainty and actual error across binned samples. We first min-max normalize both uncertainty and MSE to $[0, 1]$, then partition samples into 10 equal-width bins by normalized uncertainty. We calculate ECE as:

$$\text{ECE} = \sum_{b=1}^{10} \frac{|\mathcal{B}_b|}{N} |\bar{u}_b - \bar{e}_b| \tag{10}$$

Where $\mathcal{B}_b$ is the set of samples in bin b , N is the total number of samples, and $\bar{u}_b$, $\bar{e}_b$ are the mean normalized uncertainty and mean normalized error within bin b . A perfectly calibrated model achieves $\text{ECE} = 0$, meaning its uncertainty directly tracks its error in every bin.

Table 8: Expected calibration error (ECE).

Dataset	BioJEPAC v0.6
<i>Overall</i>	<i>0.1993</i>
Adamson	0.0873
Replogle K562 essential	0.0828
Replogle K562 genome-wide	0.2142
Norman	0.2046
Sciplex	0.3218

⁷https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_eval_uncertainty_calibration.ipynb

Perturbation Level Uncertainty-Error Pearson Correlation. We aggregate uncertainty and MSE to the perturbation level by taking the mean across all samples for each perturbation. We then calculate Pearson’s correlation between per-perturbation mean uncertainty and mean MSE:

$$r_{\text{unc}} = \frac{\sum_{p=1}^P (u_p - \bar{u})(e_p - \bar{e})}{\sqrt{\sum_{p=1}^P (u_p - \bar{u})^2} \cdot \sqrt{\sum_{p=1}^P (e_p - \bar{e})^2}} \quad (11)$$

Where u_p and e_p are the mean uncertainty and mean MSE for perturbation p , already averaged over all samples of that perturbation. The bars ($\bar{u}$, $\bar{e}$) are means across perturbations for centering. Perturbation-level aggregation smooths out sample noise, so a meaningful uncertainty signal should correlate more strongly at this level than at the sample level.

Table 9: Perturbation-level uncertainty-error Pearson correlation.

Dataset	BioJEPA-AC v0.6
<i>Overall</i>	<i>0.4037</i>
Adamson	0.6316
Replogle K562 essential	0.2597
Replogle K562 genome-wide	0.2541
Norman	0.3082
Sciplex	−0.8199

2.7 MECHANISM OF ACTION (MOA) MATCHING

We use this eval to assess whether perturbations affecting the same biological pathway produce similar changes in both the latent representation and predicted expression. This tests whether our model has learned pathway-level biology, not individual perturbation effects alone. We map each perturbation to its target gene, assign that gene to one or more pathways using KEGG Pathways 2026 (Kanehisa & Goto, 2000), and compute pairwise cosine similarity between the perturbation-level mean *latent representation* vectors:

$$\text{sim}(z_i, z_j) = \frac{\sum_{d=1}^D z_{i,d} \cdot z_{j,d}}{\sqrt{\sum_{d=1}^D z_{i,d}^2} \cdot \sqrt{\sum_{d=1}^D z_{j,d}^2}} \quad (12)$$

Where z_i and z_j are the mean predicted latent delta vectors for perturbations i and j , and D is the latent dimensionality.

For the following evaluation, we explain how the calculation is done and BioJEPA-AC’s performance by dataset. For a detailed breakdown on how the evaluations are calculated, see our explainer notebook.⁸

Within/Between Pathway Similarity Ratio. We classify each pairwise similarity as *within* (the two perturbations share at least one common pathway) or *between* (no common pathway). We take the mean of each group, compute their ratio, and test for statistical significance using a Mann-Whitney U test:

$$\begin{aligned} \text{ratio} &= \frac{\bar{s}_{\text{within}}}{\bar{s}_{\text{between}}} \\ U &= \sum_{i=1}^{n_w} \sum_{j=1}^{n_b} \mathbb{I}[s_{\text{within},i} > s_{\text{between},j}] \\ z &= \frac{U - \frac{n_w \cdot n_b}{2}}{\sqrt{\frac{n_w \cdot n_b \cdot (n_w + n_b + 1)}{12}}} \\ p &= 1 - \Phi(z) \end{aligned} \quad (13)$$

⁸https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_eval_moa_matching.ipynb

Where n_w and n_b are the number of within-pathway and between-pathway pairs, U counts how many times a within-pathway similarity exceeds a between-pathway similarity, and Φ is the standard normal CDF. A ratio above 1.0 with a small p -value indicates our model has learned to group same-pathway perturbations with statistical significance.

Table 10: Within/between pathway similarity ratio and Mann-Whitney p -value.

Dataset	Similarity Ratio	Mann Whitney p
Replogle K562 essential	1.0012	0.5547
Replogle K562 genome-wide	0.9996	0.0968

2.8 COMBINATION PERTURBATION IMPACT

We use this eval to assess whether our model has learned how multiple perturbations interact to shift a cell state beyond simple additive effects. We compare our model’s predictions for combination perturbation samples against an *additive baseline*: the sum of each individual perturbation’s real expression impact. We also use known genetic interaction maps (Norman et al., 2019; Roohani et al., 2024) to analyze results across interaction subgroups. This evaluation uses only the Norman dataset, which is our only multi-perturbation dataset. We define the additive baseline as:

$$\delta_k^{\text{add}} = \sum_{j=1}^J \delta_k^{(j)} \quad (14)$$

Where $\delta_k^{(j)}$ is the real single-perturbation expression delta for gene k of the j -th perturbation in the combination, and J is the number of perturbations (up to 4).

For the following evaluations, we explain how the calculation is done and BioJEPA-AC’s performance. For a detailed breakdown on how the evaluations are calculated, see our explainer notebook.⁹

Pearson Correlation vs Additive Baseline. For each combination perturbation, we calculate Pearson’s correlation between the mean *predicted delta* and mean *real delta* across all genes. We do the same between the additive baseline and the mean *real delta*, then report the mean Pearson across all combination perturbations for each. Since Pearson is magnitude-invariant and most genes change minimally, the large number of near-zero genes dominates this metric.

Table 11: Pearson correlation vs additive baseline (Norman dataset).

Metric	BioJEPA-AC v0.6
Model Pearson delta	0.4037
Additive baseline Pearson delta	0.7814

Beat Rate Against Additive Baseline. We calculate per-combination-perturbation MSE between the *predicted delta* and *real delta*, and between the additive baseline and *real delta*:

$$\begin{aligned} \text{MSE}_{\text{model}} &= \frac{1}{K} \sum_{k=1}^K (\hat{\delta}_k - \delta_k)^2 \\ \text{MSE}_{\text{additive}} &= \frac{1}{K} \sum_{k=1}^K (\delta_k^{\text{add}} - \delta_k)^2 \end{aligned} \quad (15)$$

⁹https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_eval_comb_perturbation.ipynb

We then compute the rate at which our model beats the additive baseline across all combination perturbations:

$$\text{beat rate} = \frac{1}{C} \sum_{c=1}^C \mathbb{I}[\text{MSE}_c^{\text{model}} < \text{MSE}_c^{\text{add}}] \quad (16)$$

Where C is the number of combination perturbations. MSE is dominated by genes with large expression changes, making this metric sensitive to DEGs. A beat rate near 0 indicates the additive baseline outperforms our model on nearly all combinations.

Table 12: Beat rate against additive baseline (Norman dataset).

Metric	BioJEPA-AC v0.6
Beat rate	0.0

Non-Additive Gene MSE (Top 20). We focus on genes where the real expression deviates most from the additive expectation, isolating non-additive (genetic interaction) effects. For each combination perturbation, we compute the deviation from the additive baseline and select the top 20 genes:

$$\mathcal{N}_c = \text{top-20 genes by } |\delta_k - \delta_k^{\text{add}}| \quad (17)$$

We then compute MSE between the *predicted delta* and *real delta* on these selected genes:

$$\text{MSE}_{\text{non-add}} = \frac{1}{|\mathcal{N}_c|} \sum_{k \in \mathcal{N}_c} (\hat{\delta}_k - \delta_k)^2 \quad (18)$$

Where $\mathcal{N}_c$ is the set of 20 genes with the largest deviation from the additive baseline for combination c . This metric tests whether our model can predict expression changes at genes where genetic interactions cause the observed effect to differ from what individual perturbation effects would predict.

Table 13: Non-additive gene MSE on top 20 (Norman dataset).

Metric	BioJEPA-AC v0.6
Non-additive top 20 MSE	0.2341

2.9 ADDITIONAL EVALUATIONS

Within each category, we run additional evaluations beyond what we report here. We also have two additional categories, Perturbation Retrieval and Dose Response, that we do not cover in this report. We do not currently pass dosage to our model, so dose response mainly validates that our model does not treat different dosages as different perturbations. Our explainer notebooks detail how each metric is calculated for those interested in the full set of analyses.

3 TRAINING

Our model comprises three separately trained components, each with its own loss function. Across all stages, we use OneCycleLR scheduling, which linearly warms up the learning rate from near-zero to the max over the first 5% of steps, then cosine-decays it back down to near-zero. We also reshuffle training data shards each epoch to prevent learning from sequential ordering. For a detailed breakdown of training loss, see our explainer notebook.¹⁰

¹⁰https://github.com/GPTomics/biojepa/blob/main/layer_explainers/explainer_training_loss.ipynb

3.1 CELL STATE ENCODER TRAINING

The first component we train is the cell state encoder, which serves as both the context encoder (student) and target encoder (teacher). The encoders learn a shared embedding space, and we freeze them after this stage. We train on 3,266,560 samples containing the cell expression profile across 10,000 genes and a total expression count. While we only have ~ 2.9 M unique cells, we upsample each dataset so that no single dataset is less than 10% of the total set. This means Adamson repeats 4 times per epoch, and Replogle K562 essential and Norman each repeat 2 times per epoch. We train for 50 epochs.

We use masked training, masking 76.6% of the input data (identified via Bayesian optimization). The target encoder starts as a copy of the context encoder. During training, the student receives a masked cell state while the teacher sees the complete unmasked cell state and provides stable target latents. We update the teacher’s weights using the exponential moving average of the student’s weights rather than gradients, preventing representation collapse and smoothing the training signal.

We use a combination of L1 loss and VICReg (Bardes et al., 2022) loss. We calculate training loss as:

$$\mathcal{L}_{\text{encoder}} = \underbrace{\lambda_{\text{sim}} \cdot \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|}_{\text{L1 reconstruction}} + \underbrace{\lambda_{\text{std}} \cdot \frac{1}{d} \sum_{j=1}^d \text{ReLU}(1 - \sigma_j)}_{\text{std loss}} + \underbrace{\lambda_{\text{cov}} \cdot \frac{1}{d} \sum_{i \neq j} C_{ij}^2}_{\text{cov loss}} \quad (19)$$

The masked predictor uses 2 transformer layers. The L1 term, weighted by $\lambda_{\text{sim}} = 40.5$, measures how accurately it reconstructs the teacher’s latents at the 76.6% of masked gene positions. The std and cov terms regularize the embedding space with $\lambda_{\text{std}} = 40.5$ and $\lambda_{\text{cov}} = 1.62$: std penalizes any feature dimension whose variance drops below 1, and cov penalizes correlations between feature dimensions, together preventing representation collapse.

3.2 ACTION COMPOSER TRAINING

The second component we train, which can run in parallel with the encoder, is the action composer. The composer learns a unified embedding space for perturbation modalities and modes of action. We train on 10,797 perturbations, each consisting of a sequence and target encoding. While BioJEPAC can handle sequence-only or target-only perturbations for inference, contrastive learning requires both, so we can only train on sequence-target pairs. We do not upsample but train for 10,000 epochs. Each perturbation is trained independently, even when datasets contain multi-perturbation samples.

We identified optimal parameters using Bayesian optimization. Since our perturbation data fits in a single shard, we reshuffle samples each epoch to vary the negative pairs within each batch. During AC predictor training, we also update the composer weights at 10% of the predictor’s learning rate to fine-tune based on prediction accuracy. We apply gradient clipping (norm=1.0) for stability.

We use Information Noise-Contrastive Estimation (InfoNCE) (van den Oord et al., 2018) loss: the cross-entropy over a temperature-scaled cosine similarity matrix between the sequence and target path representations. We calculate training loss as:

$$\mathcal{L}_{\text{align}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\hat{z}_{\text{seq},i} \cdot \hat{z}_{\text{target},i} / \tau)}{\sum_{j=1}^B \exp(\hat{z}_{\text{seq},i} \cdot \hat{z}_{\text{target},j} / \tau)} \quad (20)$$

Where $\hat{z}_{\text{seq}}$ and $\hat{z}_{\text{target}}$ are L2-normalized representations from the sequence and target paths, and $\tau = 0.012$ is the temperature. The low temperature sharpens the similarity distribution, forcing the model to strongly prefer the correct pair over all other perturbations in the batch.

3.3 ACTION CONDITIONED PREDICTOR TRAINING

After training the encoder and composer, we train the action conditioned (AC) predictor. The predictor learns to traverse the cell state latent space. We train on 3,394,560 examples consisting of the perturbation, the perturbed cell, and the control cell. While we only have ~ 2.5 M unique pairs, we upsample each dataset so that no single dataset is less than 20% of the total set. This means

Adamson repeats 7 times per epoch, Replogle K562 essential repeats 3 times, and Norman repeats 9 times per epoch. We train for 20 epochs.

We freeze both encoders and train the predictor and action composer together, with the composer receiving 10% of the predictor’s learning rate. We apply gradient clipping (norm=1.0) for stability. We use two annealing schedules: beta-NLL annealing ramps β from 0 to 0.2 over the first 30% of steps, letting the model learn accurate means before being penalized for miscalibrated confidence. Mask annealing reduces the mask ratio from 76.6% down to 10% over the final 15% of training, gradually giving the predictor more context.

We use a combination of beta-weighted Gaussian NLL (Seitzer et al., 2022) loss and VICReg (Bardes et al., 2022) loss (same role as in encoder training). We calculate training loss as:

$$\mathcal{L}_{\text{predictor}} = \underbrace{\lambda_{\text{sim}} \cdot \frac{1}{n} \sum_{i=1}^n \sigma_i^{2\beta} \cdot \frac{1}{2} \left(\log \sigma_i^2 + \frac{(y_i - \mu_i)^2}{\sigma_i^2} \right)}_{\beta\text{-weighted Gaussian NLL}} + \underbrace{\lambda_{\text{std}} \cdot \frac{1}{d} \sum_{j=1}^d \text{ReLU}(1 - \sigma_j)}_{\text{std loss}} + \underbrace{\lambda_{\text{cov}} \cdot \frac{1}{d} \sum_{i \neq j} C_{ij}^2}_{\text{cov loss}} \quad (21)$$

The Gaussian NLL term, weighted by $\lambda_{\text{sim}} = 40.5$, penalizes both inaccurate mean predictions and miscalibrated confidence at masked gene positions. The $\sigma_i^{2\beta}$ weighting down-weights high-uncertainty predictions, preventing the model from inflating variance to trivially reduce loss. Here σ_i in the NLL term is the per-element predicted standard deviation, while σ_j in the std loss is the standard deviation of feature dimension j across the batch. The std and cov terms use the same coefficients as encoder training ($\lambda_{\text{std}} = 40.5$, $\lambda_{\text{cov}} = 1.62$).

4 DATA PREPARATION

4.1 DATA SOURCES

We train BioJEPAC-AC on six publicly available single-cell perturbation datasets spanning three perturbation types (CRISPRi, CRISPRa, chemical), both single and multi-perturbation applications, and four cell types (K562, RPE1, A549, MCF7). Together, these provide ~ 3.5 million cells covering 10,324 unique perturbations. Our primary dataset is the Replogle K562 essential screen (Replogle et al., 2022), which provides the GEARS-standard (Roohani et al., 2024) held-out splits we use as our evaluation starting point.

Table 14: Dataset summary. *Encoder Training column indicates which cells are used: “All” means controls + perturbed cells; “Perturbation Only” means only perturbed cells (controls were missing due to a v0.6 data prep bug).

Dataset	Mode & Cell Type	Total Cells	Unique Perts	Batches	Genes in 10K	Encoder Training*	AC Predictor Training
Replogle K562 essential	CRISPRi K562	310,385	1,088	48	8,563	Perturbation Only	All
Replogle RPE1 essential	CRISPRi RPE1	247,914	2,393	56	8,162	Perturbation Only	None
Replogle K562 genome-wide	CRISPRi K562	1,989,578	9,866	267	8,248	All	All
Adamson	CRISPRi K562	65,337	114	1	9,749	All	All
Norman	CRISPRa K562	111,445	236	8	9,865	All	All
Sciplex	Chemical A549, K562, MCF7	799,317	188	52	9,974	All	All

Due to data prep bugs in v0.6, both the RPE1 and K562 essential control cells were missing from encoder training, and the full RPE1 dataset was missing from AC predictor training. Additionally, bugs in matching control cells impacted the Norman dataset.

4.2 GENE UNIVERSE

We define a unified gene space of 10,000 genes across all datasets. Since not every dataset measures the same genes, we use a priority-based and expression-level selection strategy. We start with all 8,563 genes from the Replogle K562 essential dataset. We then filter genes in the remaining datasets to those expressed in at least 10 cells, pool the candidates, and fill the remaining 1,437 positions with the most broadly expressed genes ranked by total cell count across datasets.

Since the datasets measure different gene panels, we generate per-dataset gene masks that record which genes each dataset actually measured. Genes outside a dataset’s panel are zero-filled in the expression vector and, currently, treated the same as no-expression genes.

4.3 EXPRESSION NORMALIZATION

We normalize raw count data using counts-per-ten-thousand (CP10K) followed by $\log_{1p}$:

$$\tilde{x}_g = \log \left(1 + \frac{x_g}{\sum_{g'=1}^G x_{g'}} \cdot 10,000 \right) \quad (22)$$

Where x_g is the raw count for gene g and G is the total number of genes. This corrects for sequencing depth differences while the log transform compresses the heavy-tailed count distribution, so the loss treats high and low expression genes more equally. We also store $\log(1 + \sum_g x_g)$ as a separate total expression input to the encoder. We exclude cells from perturbations missing both sequence and target embeddings, and filter cells with fewer than 50 expressed genes.

4.4 CONTROL MATCHING

For each dataset, we build a control bank by identifying untreated cells (labeled as control, non-targeting, or similar) and grouping them by experimental batch. For each perturbed cell, we pair it with random control cells from its batch, or if batch is unavailable, a random control from the full pool.

4.5 TRAIN, VALIDATION, AND TEST SPLITS

We split data by perturbation, not by cell, preventing leakage of both perturbation identity and cell state information. The holdout strategy differs by training stage: for encoder training, we withhold only the perturbed cells linked to held-out perturbations but train on all controls and non-held-out perturbed cells. For the action composer, perturbations are held out based on the same target list. For AC predictor training, the full perturbed cell, matched control cell, and perturbation are all held out.

We define held-out perturbations starting from the GEARS-defined (Roohani et al., 2024) split for Replogle K562 essential (734 train, 82 validation, 272 test), giving us a direct comparison to published benchmarks. For full details on the cross-dataset split strategy, see Section 2.3.

4.6 PERTURBATION EMBEDDINGS

We pass perturbation sequences and targets through publicly available foundation models during data preparation, producing the pre-computed embeddings our action composer takes as input.

4.6.1 SEQUENCE EMBEDDINGS

For genetic perturbations, the sequence is the 20-nucleotide protospacer (guide RNA target) used in the CRISPR experiment. We embed each protospacer using the Nucleotide Transformer v3 (650M parameters) (Dalla-Torre et al., 2024), producing 1,536-dimensional vectors. For CRISPRi experiments with dual-guide designs (sgID_A and sgID_B), we embed each guide separately and average. For Norman CRISPRa dual-gene perturbations, we similarly embed each gene’s protospacer and average.

For chemical perturbations, the sequence is the drug’s SMILES string. We look up SMILES for each of the 188 Sciplex compounds via PubChem, then embed them using ChemMRL (Cortes, 2025), producing 768-dimensional vectors that we zero-pad to 1,536 to share the sequence path with DNA embeddings.

In total, we generate 11,643 DNA embeddings and 188 chemical embeddings.

4.6.2 TARGET EMBEDDINGS

The target is the protein product of the perturbed gene (for genetic perturbations) or the drug’s known protein target (for chemical perturbations). If a perturbation has multiple targets, we currently take the first. We map gene identifiers to UniProt accessions via MyGene.info, retrieve protein sequences from the UniProt human proteome, and embed them using ESM-2 (8M parameters) (Lin et al., 2023), producing 320-dimensional vectors. For genes missing from UniProt, we fall back to Entrez and Ensembl protein lookups. We successfully map 9,975 of 9,985 target genes to protein sequences, with the unmapped 10 being non-coding RNA targets.

4.6.3 ALIGNMENT PAIRS

For action composer training, we need paired (sequence, target) examples where both embeddings exist. This results in 10,710 perturbations from CRISPRi screens, 98 from Norman dual-gene, 232 from Norman CRISPRa, and 188 from Sciplex chemicals, totaling 11,228 pairs split into 10,797 train, 108 validation, and 323 test.

4.6.4 MISSING EMBEDDING HANDLING

Some perturbations lack one or both embeddings. For AC predictor training, we require either sequence or target, and the action composer handles the missing path via pass-through. We exclude the 4 perturbations missing both embeddings entirely.

4.7 SHARD GENERATION

We convert the processed data into compressed NumPy archives (shards) for streaming during training. We generate separate shards for encoder, action composer, and AC predictor training since each stage requires different data layouts. Encoder and predictor shards contain 2,560 cells each.

REFERENCES

- Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.
- Eduardo Cortes. ChemMRL: A chemical matryoshka representation learning framework. <https://huggingface.co/Derify/ChemMRL>, 2025.
- Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Grzywaczewski Henault, Asa Ober-Reynolds, et al. The nucleotide transformer: Building and evaluating robust foundation models for human genomics. *Nature Methods*, 2024.
- Atray Dixit, Oren Parnas, Biyu Li, Jenny Chen, Charles P Fulco, Justus Jerber, Jefim Lindenbaum, Manu Sur, Marcin Tabaka, Aviv Regev, et al. Perturb-seq: Dissecting molecular circuits with scalable single-cell RNA profiling of pooled genetic screens. *Cell*, 167(7):1853–1866, 2016. doi: 10.1016/j.cell.2016.11.038.
- Minoru Kanehisa and Susumu Goto. KEGG: Kyoto encyclopedia of genes and genomes. *Nucleic Acids Research*, 28(1):27–30, 2000. doi: 10.1093/nar/28.1.27.
- Yann LeCun. A path towards autonomous machine intelligence. *Technical report*, 2022. URL <https://openreview.net/pdf?id=BZ5alr-kVsf>.
- Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- Thomas M Norman, Max A Horlbeck, Joseph M Replogle, Alex Y Ge, Albert Xu, Marco Jost, Luke A Gilbert, and Jonathan S Weissman. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365(6455):786–793, 2019.
- Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI*, 2018.

-
- Joseph M Replogle, Reuben A Saunders, Alexis N Pogson, Jeffrey A Hussmann, Alexander Lenail, Alina Guna, Lauren Mascibroda, Eric J Wagner, Karen Adelman, Gila Lithwick-Yanai, et al. Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell*, 185(14):2559–2575, 2022.
- Yusuf Roohani, Kexin Huang, and Jure Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024.
- Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. In *ICLR*, 2022.
- Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Aäron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.